

Constitution

Principios y guardrails globales del proyecto. Esto no es negociable por feature: cualquier spec de feature (`*-spec.md`) vive dentro de estas reglas. Los gates en `/gates` existen para hacer cumplir este documento de forma automática, no manual.

1. Arquitectura en capas (obligatoria)

Api -> Application -> Domain ^ | Infrastructure

- **Domain:** entidades y reglas de negocio puras. Cero dependencias de cualquier otra capa y cero dependencias de frameworks (nada de EF Core, ASP.NET, etc.).
- **Application:** casos de uso / servicios de aplicación. Depende de Domain. Es la **única** capa que puede depender de Infrastructure (para componer/registrar sus implementaciones, ej. vía un método de extensión `AddApplicationServices(...)` que la Api invoca).
- **Infrastructure:** implementaciones concretas (EF Core, DbContext, repositorios, acceso a datos externos). Implementa interfaces definidas en Application/Domain. Nadie depende de Infrastructure excepto Application.
- **Api:** controllers, DTOs de transporte, middleware, composition root. Depende de Application. **No** depende de Infrastructure ni de `Microsoft.EntityFrameworkCore` directamente.

Proyectos esperados en /src: Api, Application, Domain, Infrastructure (uno por capa, mismo nombre de assembly).

2. Prohibiciones explícitas

- Los **controllers no acceden a `DbContext` ni a ningún tipo de EF Core directamente**. Toda persistencia pasa por `Application`.
- Los controllers no contienen lógica de negocio: reciben el request, delegan a `Application` y traducen el resultado a una respuesta HTTP.
- `Domain` **no referencia** `Infrastructure` **ni** `Application` **ni** ningún paquete de EF Core.
- `Infrastructure` no es referenciada por `Api` **ni** por `Domain`.

3. Convenciones de naming

- Clases, métodos, propiedades: `PascalCase`.
- Parámetros y variables locales: `camelCase`.
- Campos privados: `_camelCase`.
- Interfaces: prefijo `I` (`IPProductRepository`).
- Los controllers **terminan en Controller** (`ProductsController`) y viven en el assembly `Api`.
- Métodos async terminan en `Async`.
- DTOs de request/response terminan en `Request / Response` (o `Dto` cuando son compartidos).

4. Manejo de errores estándar

- Todas las respuestas de error usan `ProblemDetails` (RFC 7807).
- Validación fallida -> `400 Bad Request` con el detalle por campo.
- Recurso inexistente -> `404 Not Found`.
- Conflictos de negocio (ej. nombre duplicado) -> `400 Bad Request` con detalle explicando la regla violada.
- Excepción no controlada -> `500 Internal Server Error` vía middleware global, sin exponer stack traces al cliente.

5. Cómo se hace cumplir esto

- **Arquitectura y prohibiciones (secciones 1 y 2):** reglas de NetArchTest en `/gates/Gates.Architecture.Tests`.
- **Naming (sección 3)** y reglas de estilo: `.editorconfig` en `/gates` con analyzers de Roslyn en `warning-as-error`.
- **Comportamiento funcional:** tests de aceptación en `/gates/Gates.Api.Tests`, derivados 1:1 de `verification.md`.

Si un cambio de código pasa el build pero rompe alguna de estas reglas, es un bug del harness, no una excepción a tolerar.